

Nama : Elyza Putri Rahayu

NIM : 254107020035

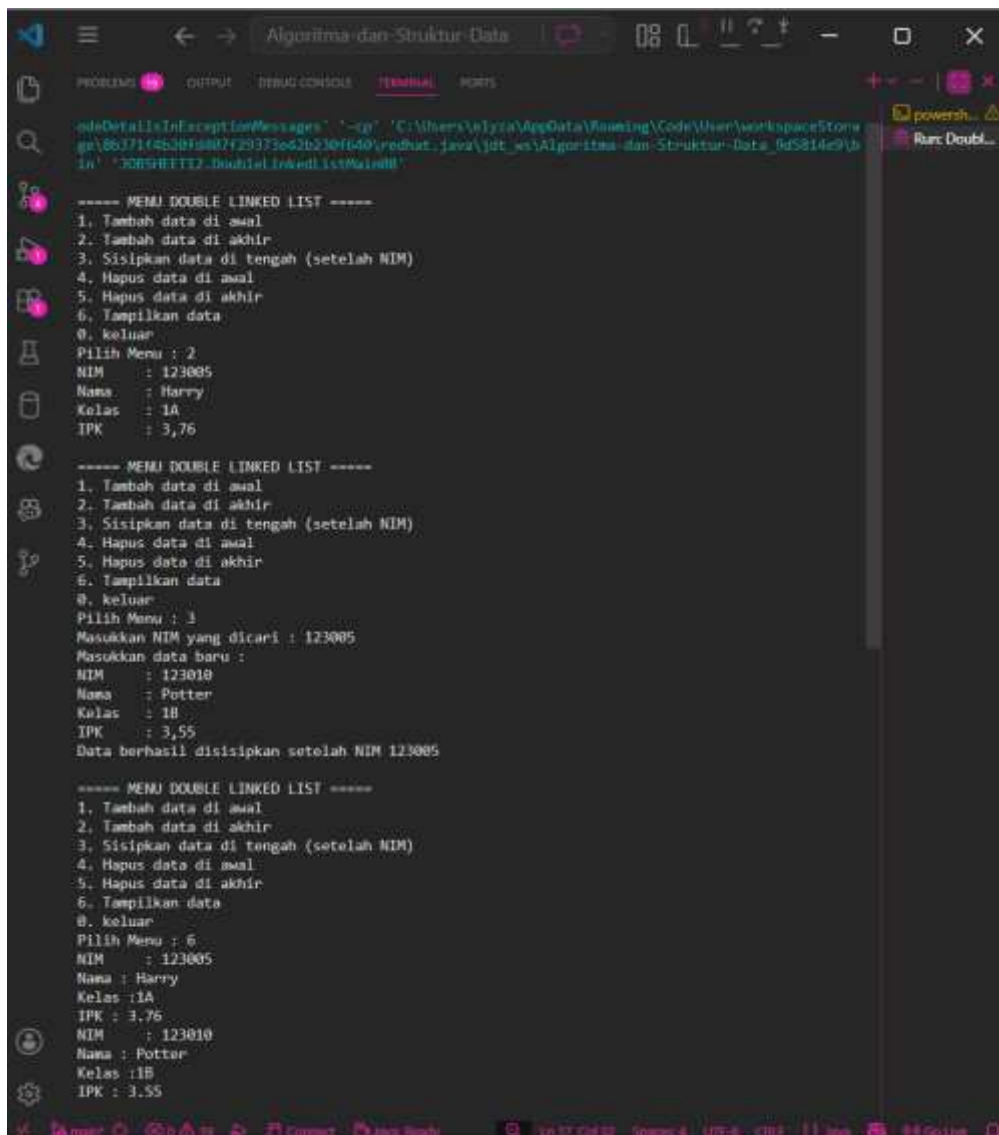
Kelas : TI – 1G

No Absen : 08

LAPORAN JOBSHEET 12

DOUBLE LINKED LIST

PERCOBAAN 1



```
odeDetailsInExceptionMessages" "-cp" "C:\Users\Elyza\AppData\Roaming\Code\User\workspaceStorage\8637144b28f8807f29373e42b230f640\redhat.java\jdk_ws\Algoritma-dan-Struktur-Data_bdS81de9\bin" "XOBSHEET12_DoubleLinkedListMainHl"

----- MENU DOUBLE LINKED LIST -----
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. keluar
Pilih Menu : 2
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3,76

----- MENU DOUBLE LINKED LIST -----
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. keluar
Pilih Menu : 1
Masukkan NIM yang dicari : 123005
Masukkan data baru :
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3,55
Data berhasil disisipkan setelah NIM 123005

----- MENU DOUBLE LINKED LIST -----
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. keluar
Pilih Menu : 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3,76
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3,55
```

Kode Program :

```
package JOBSHEET12;
```

```
public class Mahasiswa08 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    public Mahasiswa08(String nim, String nama, String kelas, double ipk) {
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.kelas = kelas;
```

```
        this.ipk = ipk;
```

```
    }
```

```
    public void tampil() {
```

```
        System.out.println("NIM    : " + nim + "\nNama : " + nama + "\nKelas : " + kelas +  
"\nIPK : " + ipk);
```

```
    }
```

```
}
```

```
package JOBSHEET12;
```

```
public class Node08 {  
  
    Mahasiswa08 data;  
  
    Node08 prev;  
  
    Node08 next;  
  
  
    public Node08(Mahasiswa08 data) {  
  
        this.data = data;  
  
        this.prev = null;  
  
        this.next = null;  
  
    }  
}
```

```
package JOBSHEET12;
```

```
public class DoubleLinkedList08 {  
  
    Node08 head;  
  
    Node08 tail;  
  
  
    public DoubleLinkedList08() {  
  
        head = null;  
  
        tail = null;  
  
    }  
  
  
    public boolean isEmpty() {
```

```
    return head == null;
}
```

```
public void addFirst(Mahasiswa08 data) {
    Node08 newNode08 = new Node08(data);
    if (isEmpty()) {
        head = tail = newNode08;
    } else {
        newNode08.next = head;
        head.prev = newNode08;
        head = newNode08;
    }
}
```

```
public void addLast (Mahasiswa08 data) {
    Node08 newNode08 = new Node08(data);
    if (isEmpty()) {
        head = tail = newNode08;
    } else {
        tail.next = newNode08;
        newNode08.prev = tail;
        tail = newNode08;
    }
}
```

```

public void insertAfter(String keyNim, Mahasiswa08 data) {

    Node08 current = head;

    while (current != null && !current.data.nim.equals(keyNim)) {

        current = current.next;

    }

    if (current == null) {

        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");

        return;

    }

    Node08 newNode08 = new Node08(data);

    // jika current adalah tail, node baru ditambahkan diakhir

    if (current == tail) {

        newNode08.prev = current;

        current.next = newNode08;

        tail = newNode08;

    } else { //node baru disisipkan ditengah

        newNode08.prev = current;

        newNode08.next = current.next;

        current.next.prev = newNode08;

        current.next = newNode08;

    }
}

```

```
        System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);  
    }
```

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong");  
        return;  
    }  
}
```

```
    Node08 current = head;  
    while (current != null) {  
        current.data.tampil();  
        current = current.next;  
    }  
}
```

```
public void removeFirst() {  
  
}
```

```
public void removeLast() {  
  
}  
}
```

```
package JOBSHEET12;
```

```
import java.util.Scanner;
```

```
public class DoubleLinkedListMain08 {
```

```
    public static Mahasiswa08 inputMahasiswa08(Scanner sc) {
```

```
        System.out.print("NIM\t: ");
```

```
        String nim = sc.nextLine();
```

```
        System.out.print("Nama\t: ");
```

```
        String nama = sc.nextLine();
```

```
        System.out.print("Kelas\t: ");
```

```
        String kelas = sc.nextLine();
```

```
        System.out.print("IPK\t: ");
```

```
        double ipk = sc.nextDouble();
```

```
        sc.nextLine();
```

```
        return new Mahasiswa08(nim, nama, kelas, ipk);
```

```
    }
```

```
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

DoubleLinkedList08 list = new DoubleLinkedList08();

int pilihan;

do {

    System.out.println("\n===== MENU DOUBLE LINKED LIST =====");

    System.out.println("1. Tambah data di awal");

    System.out.println("2. Tambah data di akhir");

    System.out.println("3. Sisipkan data di tengah (setelah NIM)");

    System.out.println("4. Hapus data di awal");

    System.out.println("5. Hapus data di akhir");

    System.out.println("6. Tampilkan data");

    System.out.println("0. keluar");


    System.out.print("Pilih Menu : ");

    pilihan = sc.nextInt();

    sc.nextLine();

    switch (pilihan) {

        case 1:

            Mahasiswa08 mhsAwal = inputMahasiswa08(sc);

            list.addFirst(mhsAwal);

            break;

```

case 2:

```
Mahasiswa08 mhsAkhir = inputMahasiswa08(sc);
```

```
list.addLast(mhsAkhir);
```

```
break;
```

case 3:

```
System.out.print("Masukkan NIM yang dicari : ");
```

```
String keyNim = sc.nextLine();
```

```
System.out.println("Masukkan data baru : ");
```

```
Mahasiswa08 dataBaru = inputMahasiswa08(sc);
```

```
list.insertAfter(keyNim, dataBaru);
```

```
break;
```

case 4:

```
list.removeFirst();
```

```
break;
```

case 5:

```
list.removeLast();
```

```
break;
```

case 6:

```
list.print();
```

```

        break;

    case 0:

        System.out.println("program selesai.");

        break;

    default:

        System.out.println("Menu tidak valid");

    }

    } while (pilihan != 0);

    sc.close();

}

}

```

PERTANYAAN PERCOBAAN 1

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawab : Single Linked List hanya memiliki satu pointer yaitu next, sehingga traversal data hanya bisa dilakukan dari depan ke belakang. Sedangkan Double Linked List memiliki dua pointer yaitu next dan prev, sehingga traversal dapat dilakukan dua arah, yaitu dari head ke tail maupun dari tail ke head. Karena memiliki dua pointer, proses manipulasi data seperti penyisipan dan penghapusan node menjadi lebih mudah dibanding Single Linked List.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawab : Atribut next digunakan untuk menunjuk node setelah node saat ini, sehingga traversal bisa bergerak maju ke node berikutnya. Sedangkan atribut prev digunakan untuk menunjuk node sebelumnya, sehingga traversal dapat bergerak mundur ke node sebelumnya. Kedua atribut ini membuat Double Linked List dapat ditelusuri dua arah.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawab : Konstruktor digunakan untuk menginisialisasi kondisi awal linked list agar masih kosong. Pada kondisi awal, head dan tail bernilai null karena belum ada node yang disimpan di dalam linked list.

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode08;
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawab : Karena saat linked list masih kosong dan data pertama dimasukkan, maka node tersebut menjadi satu-satunya node dalam linked list. Oleh sebab itu head dan tail harus menunjuk node yang sama, karena node tersebut sekaligus menjadi node pertama dan node terakhir.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

Jawab : Sudah dimodifikasi pada kode berikut:

```
if (isEmpty()) {  
    System.out.println("Linked List masih kosong");  
}
```

6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawab :

```
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong");  
    } else {  
        Node08 temp = tail;  
  
        while (temp != null) {  
            temp.data.tampil();  
            temp = temp.prev;  
        }  
    }  
}
```

```
}
```

PERCOBAAN 2

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. keluar
Pilih Menu : 5
Data Berhasil Dihapus.

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. keluar
Pilih Menu : 6
NIM      : 123005
Nama     : Harry
Kelas  :1A
IPK     : 3.76
```

Kode Program :

```
public void removeFirst() {

    if (isEmpty()) {

        System.out.println("Linked List kosong");

        return;

    }

    if (head == tail) {
```

```

        head = tail = null;

    } else {

        head = head.next;

        head.prev = null;

    }

}

public void removeLast() {

    if (isEmpty()) {

        System.out.println("Linked List kosong");

        return;

    }

    if (head == tail) {

        head = tail = null;

    } else {

        tail = tail.prev;

        tail.next = null;

    }

}

}

```

PERTANYAAN PERCOBAAN 2

1. Perhatikan potongan kode berikut pada method
removeFirst(): head = head.next;

```
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawab : Baris `head = head.next;` digunakan untuk memindahkan posisi head ke node berikutnya setelah node pertama dihapus.

Sedangkan `head.prev = null;` digunakan untuk menghapus hubungan node baru dengan node sebelumnya agar node baru benar-benar menjadi node paling depan dan tidak lagi terhubung dengan node yang sudah dihapus.

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

Jawab :

```
public void removeFirst() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong");  
    } else if (head == tail) {  
  
        System.out.println("Data yang dihapus : ");  
        head.data.tampil();  
  
        head = tail = null;  
  
    } else {  
  
        System.out.println("Data yang dihapus : ");  
        head.data.tampil();  
  
        head = head.next;  
        head.prev = null;  
    }  
}  
  
public void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong");  
    }  
}
```

```
} else if (head == tail) {  
  
    System.out.println("Data yang dihapus : ");  
    tail.data.tampil();  
  
    head = tail = null;  
  
} else {  
  
    System.out.println("Data yang dihapus : ");  
    tail.data.tampil();  
  
    tail = tail.prev;  
    tail.next = null;  
}  
}  
}
```